

C2000 T-Format Encoder Interface Software Guide

SDK v5.00.00

CONTENTS

1	Getting Started	1
1.1	Overview	1
1.2	Getting Started Steps	2
1.3	Migrating from an Earlier Version	2
1.4	Minimum Requirements	5
1.5	Installation and Directory Structure	6
2	Flowcharts	8
3	T-Format System Solution	10
3.1	Configuration	10
3.2	Build and Run the Example	15
3.3	System-Level Functions	22
4	T-Format Encoder Interface Library	24
4.1	T-Format Library Integration	24
4.2	T-Format Library Porting to CPU2	27
4.3	Global Variables	28
4.4	T-Format Encoder Interface API	29
4.5	Library Private Functions	37
4.6	Protocol Defines	41
4.7	Library Deprecated Functions	49
5	Additional Resources	51
5.1	Abbreviations and Acronyms	51
6	Changelog	53
6.1	C2000 MotorControl SDK Version 5.00.00.00	53
6.2	C2000 MotorControl SDK Version 4.01.00.00	53
7	Important Notice	54
8	Disclaimer	56
	Index	57

GETTING STARTED

1.1 Overview

T-Format Encoder Interface by Tamagawa Seiki, is designed for serial transfer of digital data between linear, rotary or angle encoders, touch probes, accelerometers and the subsequent electronics, such as numerical controls, servo amplifiers and programmable logic controllers.

For more details on the protocol and implementation aspects, see the [Tamagawa Seiki](#) website.

The C2000 T-Format Encoder Interface Reference is divided into three parts:

1. Hardware and overall design. This is described in [C2000 T-Format Encoder Interface TI Design \(TIDM-1011\)](#).
2. Software: T-Format encoder interface Application Programming Interface (API)
3. Software: System solution demonstration

This document focuses on the T-Format API and system example.

1.1.1 T-Format API

The T-Format API provides functions and definitions used to implement the T-Format encoder interface.

Key Features of the library:

- Encoder interface integrated into a C2000 real-time microcontroller solution
- Meets the encoder interface clocking requirements
- Supports all T-Format commands
- Cyclic Redundancy Check (CRC) calculations
- All source code is provided

1.1.2 System Example

Provides support functions to configure a C2000 Real-Time Control device to support T-format.

Key Features:

- Provides a blueprint for how to integrate the library into a project
- Command processing to perform basic error checks, update position, turns and encoder ID.
- All source code is provided.

Note: For device-specific requirements, refer to the [TIDM-1011](#) T-Format Encoder Interface design guide.

1.2 Getting Started Steps

The recommended sequence to get started is:

1. Review and install the tools as described in *the minimum requirements*.
2. Run the *system example*.
3. If implementing your own system, refer to *T-Format library integration*.

Warning: If you are migrating your application from V04.01.00.00 or earlier, *library migration* includes important information regarding changes and enhancements.

1.3 Migrating from an Earlier Version

The C2000 Motor Control SDK V4.01.00_00 release implemented important changes. These changes improve performance, increase ease-of-use, and utilize new device features.

The changes fall into two categories:

- Code Changes: Changes that require you to modify your source code.
- Design Changes: Changes that may not impact your code. These primarily use new device features, but are not required.

A specific list of changes are described in the following sections.

1.3.1 Code Changes

The suggested steps to migrate an application to the new library are:

Review the system example

Illustrates the MCU setup and invokes each of the T-format commands.

Global variable changes

Refer to *global variables*. The tformatData structure has changed slightly:

1. The tformatRxData[] array is now separate from the tformatData structure. tformatRxData and must be defined in your application.
2. tformatData variable names were updated for clarity.
3. Position and Turns were removed and added to an application level structure.

Listing 1.1: /boostxl_posmgr/f28003x/source/pm_tformat_main.c

```

//
// Globals used by the library
// - Lookup table for CRC calculations
// - tformatData structure to track key parameters.
//
PM_tformat_DataStruct tformatData;
volatile uint16_t tformatRxData[PM_TFORMAT_FIELDS_MAX];

#if defined(PM_TFORMAT_CRC_TABLE)
    uint16_t tformatCRCTable[PM_TFORMAT_CRCTABLE_SIZE];
#endif

//

```

In the application demo, the following variables have been added to collect data. They are not required by the library.

1. errorData: detected errors are flagged
2. encoderData: encoder position, turns, and id
3. commandData: Command information (command field sent, address, data) can be used by the application for error checking against the command information received.
4. tformatSpiRxIsrTicker: Increments once each time the SPI receive interrupt service routine is called.

Listing 1.2: /boostxl_posmgr/f28003x/source/pm_tformat_main.c

```

//
// These structures are used to display the received
// data in Code Composer Studio's Expressions
// window during development. Because the structure is
// set, but never used, the pragma RETAIN
// has been applied. Volatile is used for the
// tformatSpiRxIsrTicker since it is updated within
// the SPI ISR.
//
#pragma RETAIN (encoderData);
#pragma RETAIN (errorData);
#pragma RETAIN (commandData);

PM_tformat_encoderStruct encoderData;
PM_tformat_errorStruct errorData;
PM_tformat_commandStruct commandData;
volatile uint32_t tformatSpiRxIsrTicker;

//

```

SPI ISR changes in tformat_spiRxISR

1. Update the ISR to mask the data (with 0xFF) as it is read from the FIFO.
2. Update the ISR to use tformatRxData[]
3. The use of MUST_ITERATE was added to improve performance. Refer to [TMS320C28x Optimizing C/C++ Compiler User's Guide](#).
4. Refer to the system demonstration code for an example.

Listing 1.3: /boostxl_posmgr/shared/source/tformat.c

```
//
#pragma MUST_ITERATE(5, 12, 1)
for (i = 0; i < tformatData.fifoLevel; i++)
{
    tformatRxData[i] = SPI_readDataNonBlocking(PM_TFORMAT_SPI) & 0xFF;
}
//
```

Command (request) setup changes

- PM_tformat_setupCommand() split into 3 functions based on the fields transmitted.
- Refer to *deprecated command functions*.

Receive data changes

- PM_tformat_receiveData() split into 5 functions based on the fields received.
- Refer to *deprecated command functions*.

CRC calculations moved to the library

- Application level CRC calls are no longer required and can be removed.
- The calculations are included in the setup command and the received data functions.
- The CRC result is passed to the application through the tformatData.crcCheck variable.
- The application can compare tformatData.crcCheck to tformatData.crcField (CRC received from the encoder).

New API functions

- PM_tformat_updatePositionOrTurns() and PM_tformat_updateEncoderID()
- New functions added. Previously the translation from response received was left to the application.
- Refer to *Update encoder parameters*.

1.3.2 Design Changes

These design changes are included in the library or system examples. There is no specific action to take. They are listed here for reference.

Receive CRC generation (requires CLB type 2)

Devices with Type2 CLB, or later, can calculate the RX CRC as the response from the encoder is received. The library uses this option when available. Refer to *system and library configuration instructions*. No application level change is required.

Transmit CRC generation (requires VCRC module)

When available, the library uses the VCRC module to generate the TX CRC. TX CRC is used by the EEPROM write and EEPROM read commands. Refer to *system and library configuration instructions*. No application level change is required.

CLB_SPI_CLK (out) to SPICLK (in) pin connection (requires CLB type 2)

Internal clocking of the SPICLK pin. Devices with CLB type 2, or later, are capable of connecting to the SPICLK (in) directly. Externally connecting the CLB_SPI_CLK output to the SPICLK pin input is not required. The CLB design takes advantage of this feature when available. In such a case, the SPICLK can optionally be brought out to a pin to be monitored. Refer to the input/output details in the [C2000 T-Format Encoder Interface TI Design \(TIDM-1011\)](#).

SPILOCK alignment with data

Previously the CLB_SPI_CLK was only aligned with the first falling edge of the encoder's response. LUT0 has been modified to re-align the clock when any edge (falling or rising) within the response is received.

FIFO wordsize

The FIFO word size has been aligned with the T-format field size. Previously the word size was set to 16-bits. This change simplified extracting the fields from the FIFO.

INPUTXBAR and OUTPUTXBAR usage

CLB_INPUTXBAR and CLB_OUTPUTXBAR were used in favor of device-level INPUTXBAR/OUTPUTXBAR on devices supporting this feature.

1.4 Minimum Requirements

The following development tools were used to build and test the release:

1.4.1 Development Environment

- Code Composer Studio v12.4.0
- C2000 Code Generation Tools v22.6.0.LTS
 - The code was built for abi = EABI
 - Optimization levels used for benchmarks: -o3 -mf 5
 - For more information, refer to the [C2000 Optimization Guide](#)

1.4.2 Software Development Kit

- C2000Ware Motor Control SDK v5_00_00_00
- C2000Ware for C2000 MCUs v5.00.00.00. C2000Ware is included within the Motor Control SDK.

1.4.3 Hardware

The hardware required is documented in [TIDM-1011](#). This includes:

- Supported C2000 LaunchPads
- C2000 device requirements
- The Position Manager BoosterPack (BOOSTXL-POSMGR)
- A T-Format encoder with an appropriate shielded cable.

Note: Build configuration for the [TMDXIDDK379D Development Kit](#) is provided for some devices. Refer to *the system example section*.

1.5 Installation and Directory Structure

1.5.1 Installing the Examples

After installing the tools described in [requirements](#), the library and example projects can be found in the following SDK directory:

1.5.2 Root Directory

The default install location is shown below:

Contents	Abbreviation	Default Location
Motor Control SDK	[MC_SDK_ROOT]	C:/ti/c2000/C2000Ware_MotorControl_SDK_<version>
System Example	[SYS_ROOT]	[MC_SDK_ROOT]/solutions/boostxl_posmgr/
API Library	[API_ROOT]	[MC_SDK_ROOT]/libraries/position_sensing/tformat

1.5.3 System Example Directory Structure

The system reference example source is located in the following directory structure with respect to [MC_SDK_ROOT]/solutions/boostxl_posmgr/:

System Reference Directory	Contents
[SYS_ROOT]/ccs/tformat/	Code Composer Studio projectspec file
[SYS_ROOT]/shared/include/tformat*.h	System Header file. Shared between devices
[SYS_ROOT]/shared/source/tformat*.c	System source code. Shared between devices
[SYS_ROOT]/shared/debug/tformat_vars.txt	CCS expressions import file.
[SYS_ROOT]/<device>/cmd	Linker command file. Device specific
[SYS_ROOT]/<device>/source/*tformat*.c	C28x C-source files. Device specific
[SYS_ROOT]/<device>/source/tformat*.syscfg	Device-specific pin-Mux and XBAR, CLB interconnect

1.5.4 Library Directory Structure

The API source is located in the following directory structure with respect to [MC_SDK_ROOT]/libraries/position_sensing/tformat:

API Library Reference	Contents
[API_ROOT]/ccs/<device>	Code Composer Studio projectspec file
[API_ROOT]/include	Header files
[API_ROOT]/source/pm_tformat_source.c	C28x BiSS-C communication C-source code
[API_ROOT]/source/pm_tformat_crc.c	C28x CRC C-source code
[API_ROOT]/source/pm_tformat_crc_vcrc.asm	VCRC module CRC assembly source code
[API_ROOT]/source/pm_tformat<device>.syscfg	CLB Tool: tile configuration. Device specific.
[API_ROOT]/lib/pm_tformat_<device>.lib	Device-specific binary

FLOWCHARTS

Fig. 2.1 shows the software flow of the system example and the processing of a command. Use these flowcharts as a reference for the rest of the documentation.

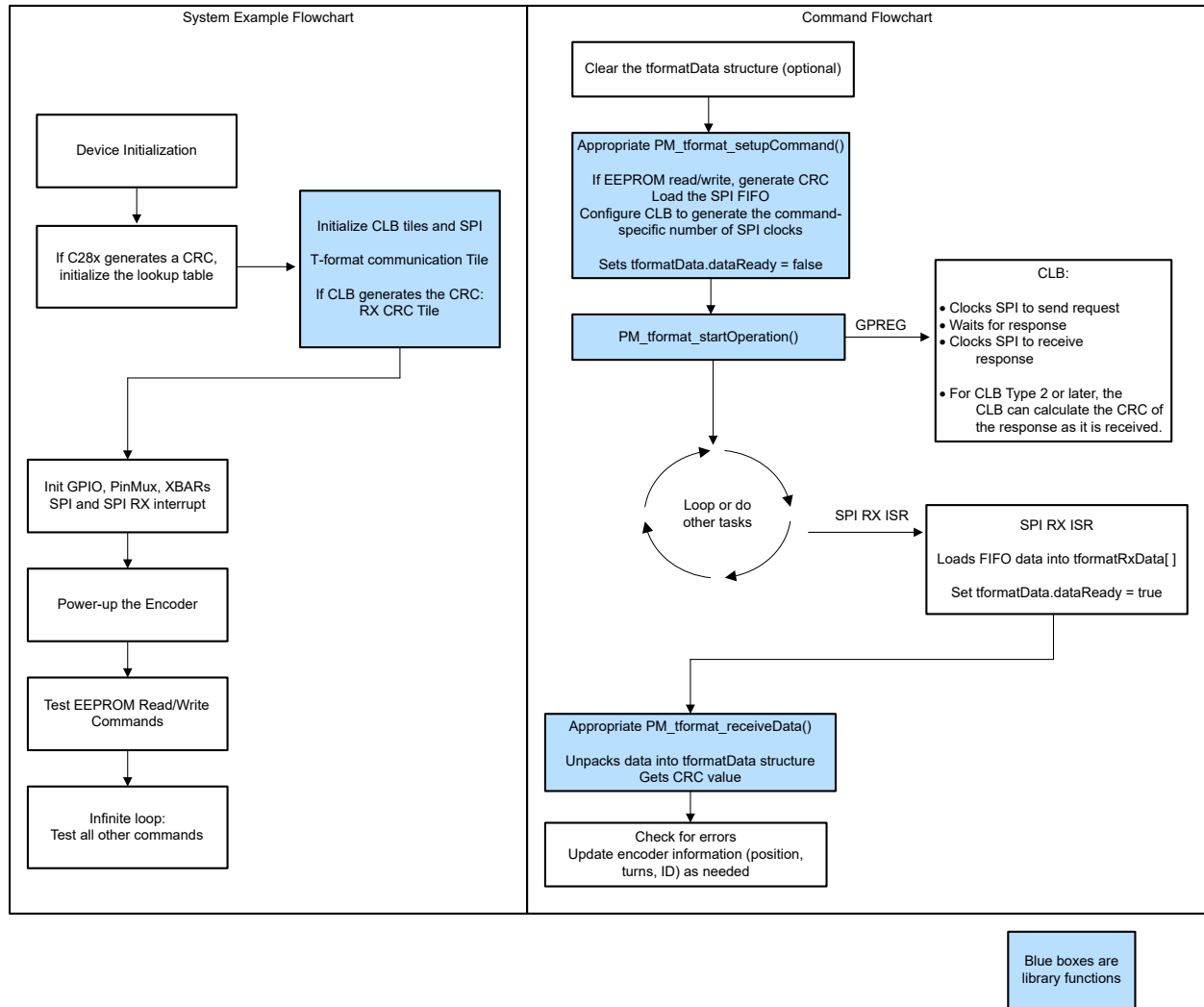


Fig. 2.1: T-Format Interface Flowcharts.

T-FORMAT SYSTEM SOLUTION

3.1 Configuration

3.1.1 System Example Configuration

If you wish to change the default configuration of the system example, edit the example's header file.

Listing 3.1: /solutions/boostxl_posmgr/shared/include/tformat.h

```
//
// Indicate which commands should be run. This can be helpful
// for debug or benchmarking
// 1 == run command
// 0 == skip command
//
//
→ *****
#define TFORMAT_RUN_ID0      1
#define TFORMAT_RUN_ID1      1
#define TFORMAT_RUN_ID2      1
#define TFORMAT_RUN_ID3      1
#define TFORMAT_RUN_IDD_ID6 1 // EEPROM read and write
#define TFORMAT_RUN_ID7      1
#define TFORMAT_RUN_ID8      1
#define TFORMAT_RUN_IDC      1

//
→ *****
//
// The duration of a single bit is 2.5 MHz (400ns)
// First determine the number of CLB clocks within
// a 400 ns period. Then divide this by 2, in order
// to create a clock within the CLB with a 400ns period.
//
// Define the number of CLB clocks required to generate for the bus clock
// Define the number of CLB clocks within a bit-width
//
//
→ *****
#if defined(_F2837X)
```

(continues on next page)

(continued from previous page)

```

#define TFORMAT_CLB_CLOCK      100u1 // MHz

#elif defined(_F2838X)
#define TFORMAT_CLB_CLOCK      100u1 // MHz

#elif defined(_F28004X)
#define TFORMAT_CLB_CLOCK      100u1 // MHz

#elif defined(_F28003X)
#define TFORMAT_CLB_CLOCK      120u1 // MHz

#elif defined(_F28002X)
#define TFORMAT_CLB_CLOCK      100u1 // MHz

#elif defined(_F28P65X)
#define TFORMAT_CLB_CLOCK      100u1 // MHz

#else
#error Specify the CLB clock rate for the specific device
#endif
#define TFORMAT_BUS_CLOCK      2.5f // MHz
#define TFORMAT_FREQ_DIVIDER   (TFORMAT_CLB_CLOCK /
→ \
                                (uint16_t) (2.0f * TFORMAT_BUS_CLOCK) )
#define TFORMAT_BIT_WIDTH      (TFORMAT_CLB_CLOCK /
→ \
                                (uint16_t) (TFORMAT_BUS_CLOCK) )

//
→ *****
//
// For EEPROM read/write testing, specify the range of addresses to be used
//
// Note: Select addresses between 0 and 126 (0x7E).
//       Address 127 (0x7F) is the EEPROM page control.
//
//
→ *****
#define TFORMAT_EEPROM_TEST_ADDR_START  0x73u
#define TFORMAT_EEPROM_TEST_ADDR_END    0x7Eu

//
→ *****
//
// Delay time, in micro-seconds, between each command sent
// The increment is used to vary the time between commands for testing
//
//
→ *****
#define TFORMAT_IDLE_DELAY_US_START      10u1
#define TFORMAT_IDLE_DELAY_US_INCREMENT  15u1

```

(continues on next page)

(continued from previous page)

```

//
→ *****
//
// Time to wait after a write to the EEPROM before another write/read
//
//
→ *****
#define TFORMAT_EEPROM_BUSY_US 18000u1

//
→ *****
//
// Power-on to normal operation delay 1.5 seconds. Refer to the data manual
// for a specific encoder.
//
//
→ *****
#define TFORMAT_POWER_ON_DELAY_US 1500000u1

//

```

3.1.2 Library Configuration

If you wish to change the default configuration of the library, edit the library's public header file.

Note:

- If you change the library configuration then recompile **both** the library **and** the system example.
- CRC generation by the CLB requires CLB type 2 or later. This will use additional CLB resources, but frees C28x bandwidth.

Listing 3.2: /libraries/position_sensing/tformat/include/pm_tformat_include.h

```

//
// #define PM_TFORMAT_SPI SPIA_BASE, SPIB_BASE, SPIC_BASE, SPID_BASE
// Define which SPI peripheral is used. This must match which instance
// the CLB is clocking and connected to the RS485 bus.
// Note: The CLB output configuration may need to be modified if this
// selection is changed. Refer to the design guide for more information.
//
// #define PM_TFORMAT_CLB_INPUTXBAR
// Define this identifier if the CLB has an INPUTXBAR and OUTPUTXBAR
// The system initialization routine will call the appropriate functions
// based on this define.
//
// [doc-library-crc-config-start]
//
// Define how the CRC is generated. Supported options are:
//

```

(continues on next page)

(continued from previous page)

```
// #define PM_TFORMAT_RX_CRC_BY_C28X
// #define PM_TFORMAT_TX_CRC_BY_C28X
//     The CRC for the data (TX or RX) is generated by the C28x using a
//     lookup table. This method takes the most C28x bandwidth and
//     memory.
//
// #define PM_TFORMAT_RX_CRC_BY_CLB
//     The received data CRC is generated on-the-fly by the CLB as the
//     data is received.
//     This requires 1 CLB tile.
//     This option requires CLB type 2 or later.
//
// #define PM_TFORMAT_TX_CRC_BY_VCRC
//     The transmit data CRC is generated by the VCRC extension to the C28x.
//     This option requires that the device has the VCRC extension.
//
// [doc-library-crc-config-end]
//
//
```

```
→ *****

#if defined(_F2837X) || defined(F2837x_DEVICE)
    #define PM_TFORMAT_SPI                SPIB_BASE
    #define PM_TFORMAT_RX_CRC_BY_C28X
    #define PM_TFORMAT_TX_CRC_BY_C28X
    #define PM_TFORMAT_USE_LEGACY_TERMINOLOGY

#elif defined(_F28002X) || defined(F28002x_DEVICE)
    #define PM_TFORMAT_SPI                SPIB_BASE
    #define PM_TFORMAT_RX_CRC_BY_CLB
    #define PM_TFORMAT_TX_CRC_BY_VCRC
    #define PM_TFORMAT_USE_CLB_INPUTXBAR
    #define PM_TFORMAT_USE_LEGACY_TERMINOLOGY

#elif defined(_F28004X) || defined(F28004x_DEVICE)
    #define PM_TFORMAT_SPI                SPIB_BASE
    #define PM_TFORMAT_RX_CRC_BY_CLB
    #define PM_TFORMAT_TX_CRC_BY_C28X
    #define PM_TFORMAT_USE_LEGACY_TERMINOLOGY

#elif defined(_F28003X) || defined(F28003x_DEVICE)
    #define PM_TFORMAT_SPI                SPIB_BASE
    #define PM_TFORMAT_RX_CRC_BY_CLB
    #define PM_TFORMAT_TX_CRC_BY_VCRC
    #define PM_TFORMAT_USE_CLB_INPUTXBAR
    #define PM_TFORMAT_USE_LEGACY_TERMINOLOGY

#elif defined(_F2838X) || defined(F2838x_DEVICE)
    #define PM_TFORMAT_SPI                SPIB_BASE
    #define PM_TFORMAT_RX_CRC_BY_CLB
    #define PM_TFORMAT_TX_CRC_BY_VCRC
    #define PM_TFORMAT_USE_CLB_INPUTXBAR
```

(continues on next page)

(continued from previous page)

```

#define PM_TFORMAT_USE_LEGACY_TERMINOLOGY

#elif defined(_F28P65X) || defined(F28P65x_DEVICE)
    #define PM_TFORMAT_SPI          SPID_BASE
    #define PM_TFORMAT_RX_CRC_BY_CLB
    #define PM_TFORMAT_TX_CRC_BY_VCRC
    #define PM_TFORMAT_USE_CLB_INPUTXBAR
    #define PM_TFORMAT_USE_INCLUSIVE_TERMINOLOGY

#else
    #error Device should match a predefined symbol in the build options.
#endif

#if(PM_TFORMAT_SPI == SPIB_BASE)
    #define PM_TFORMAT_INT_SPI_RX    INT_SPIB_RX
#elif(PM_TFORMAT_SPI == SPIA_BASE)
    #define PM_TFORMAT_INT_SPI_RX    INT_SPIA_RX
#elif(PM_TFORMAT_SPI == SPID_BASE)
    #define PM_TFORMAT_INT_SPI_RX    INT_SPID_RX
#else
    #error Update SPI code for the correct instance.
#endif

//
// Do we need to build the C28X CRC lookup table?
// Indicates C28x lookup table for a CRC calculation is being used.
//
#if(defined(PM_TFORMAT_RX_CRC_BY_C28X) || defined(PM_TFORMAT_TX_CRC_BY_C28X) )
    #define PM_TFORMAT_CRC_TABLE
#endif

//
// Do some checking of the defined configuration
//
#if(defined(_F28379D) &&
    ↪ \
        ( defined(TFORMAT_RX_BY_CLB) || defined(TFORMAT_TX_BY_CLB) ) )
{
    #error CRC by CLB is not supported on this device. Use C28x lookup table.
}
#endif
#if(defined(_F28379D) && defined(TFORMAT_TX_BY_VCRC))
{
    #error VCRC is not supported on this device. Use C28x lookup table.
}
#endif
#if(defined(_F28004X) && defined(TFORMAT_TX_BY_VCRC))
{
    #error VCRC is not supported on this device. Use C28x lookup table.
}
#endif

```

(continues on next page)

(continued from previous page)

//

3.2 Build and Run the Example

This example provides a roadmap for integration of the library into an application.

Note: For Code Composer Studio information, refer to the [Code Composer Studio User's Guide](#).

3.2.1 Hardware Connections

Setup the hardware as described in the [TIDM-1011](#) design guide.

3.2.2 Import and Run Example Project

The system example directory, `[MC_SDK_ROOT]/solutions/boostxl_posmgr/<device>`, contains a Code Composer Studio projectspec file. To import the project into CCS, follow these steps:

1. Install the minimum required tools. Refer to [Minimum Requirements](#).
2. Import the example system demonstration.
 1. In Code Composer Studio (CCS), click *Project -> import CCS Projects..*
 2. Navigate to the system example directory: `[MC_SDK_ROOT]/solutions/boostxl_posmgr/<device>`
 3. In the `ccs/tformat/` directory, select the `tformat_systemtest.projects` file
 4. Click *Finish*.
 5. In the CCS Project Explorer you should now see the demo and library projects as shown in [Fig. 3.1](#).

Note: When you import the system example, CCS will also import the associated library project.

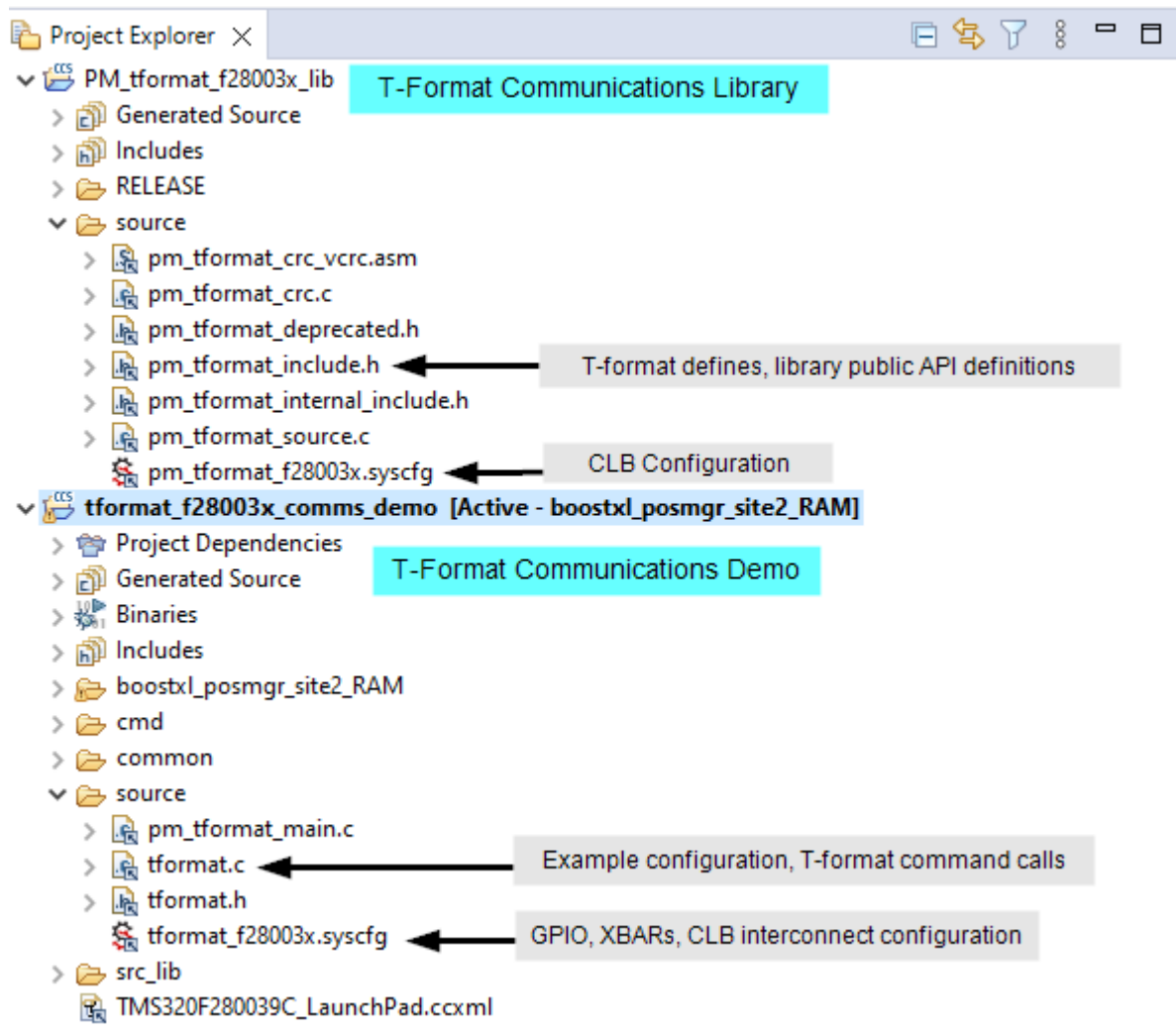


Fig. 3.1: Code Composer Studio Project Explorer Window.

3. Select the build configuration of choice

1. To select the build configuration: *right-click on the project*
2. Select: *build configurations* and *set active*.
3. Select the build configuration.

The build configurations are:

boostxl_posmgr_site2_RAM: Builds the solution to load and run from RAM. The pin-out corresponds to the LaunchPad + BOOSTXL_POSMGR site2 as described in [TIDM-1011](#) design guide.

boostxl_posmgr_site2_FLASH: The solution loads the demo to flash. At run-time, critical portions are copied to execute from RAM.

tidxiddk379d_h1_RAM: This option is available for some devices. In this case, the pinout corresponds to the H1 connector on the [TMDXIDDK379D Development Kit](#).

Note: A LaunchPad is not available for F28388x devices. The demo can be run on either

the TMDXIDDK379D Development Kit or a controlCard. For a controlCard, the user must manually connect the required pins to a BOOSTXL-POSMGR.

1. Select the active board configuration
 1. Some projects include two board configuration files (.ccxml) as shown in Fig. 3.2. This occurs when the LaunchPad and ControlCard use different JTAG connections.
 2. *Right-click* on the board configuration matching your hardware.
 3. Select *Set as active target configuration*

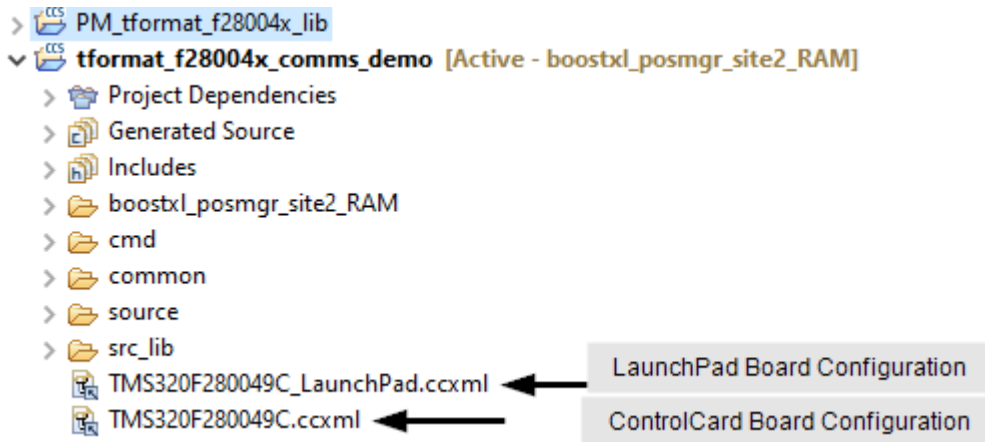


Fig. 3.2: Board Configurations

2. Build the system example project
 1. In the project manager window, *Right-click* on the project name `tformat_<device>_boostxl_posmgr_site2`
 2. Select *Rebuild Project*
 3. Observe the console window for any errors or for successful completion of the build.
 4. Once the build completes, without errors, execute the project by selecting *Run -> Debug*
3. Populate the expressions window
 1. *Right-click* on the expressions window
 2. Select *import*
 3. Browse to the location `[SYS_ROOT]/shared/debug/`
 4. Select the file `tformat_vars.txt` and click *open*
 5. At the top of the expressions window, enable *continuous refresh* as shown in Fig. 3.5.

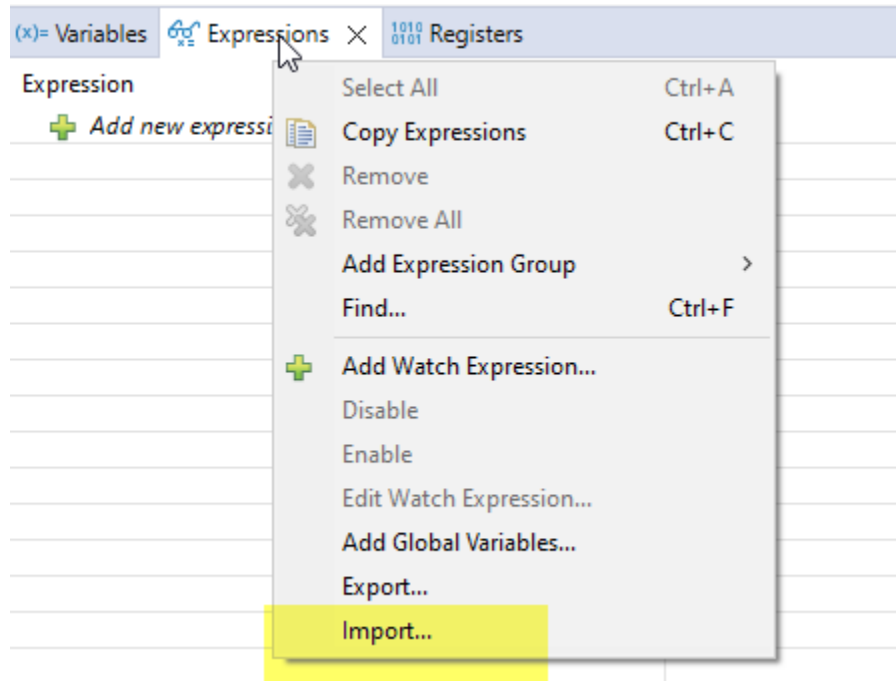


Fig. 3.3: Code Composer Studio Import Expressions.

Expression	Type	Value
> tformatData	struct PM_tformat_DataStruct	{controlFiel
> tformatRxData	unsigned int[12]	[255,64,4,13
▼ encoderData	struct PM_tformat_encoderStruct	{position=3
(x)= position	unsigned long	3571
(x)= turns	unsigned long	7
(x)= id	unsigned int	0x0011 (He
▼ errorData	struct PM_tformat_errorStruct	{idCheckErr
(x)= idCheckError	unsigned int	0
(x)= crcCheckError	unsigned int	0
(x)= eepromAddressError	unsigned int	0
(x)= eepromDataError	unsigned int	0
▼ commandData	struct PM_tformat_commandStruct	{controlFiel
(x)= controlField	unsigned long	64
(x)= eepromAddress	unsigned long	0
(x)= eepromData	unsigned int	0

Fig. 3.4: Code Composer Studio Expressions Window.

Note: The system example clears the tformatData structure between commands. For this reason, the structure may appear to be all zero when running the demo. To view the content, set a breakpoint before the structure is cleared.

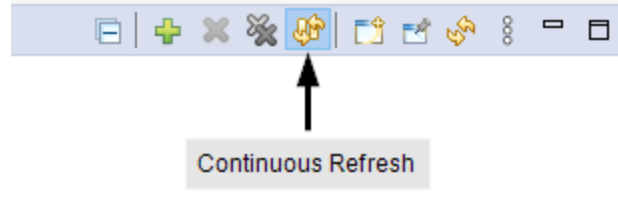


Fig. 3.5: Enable Continuous Refresh.

4. Run the code

1. Press the *Run/Resume* button
2. Manually turn the shaft of the motor or encoder
3. Monitor the signals and variables described below:

group **TFORMAT_API_DATA_STRUCT**
[TFORMAT_API_DATA_STRUCT_SNIPPET]

Information sharing structure

Variables

PM_tformat_DataStruct **tformatData**

The received T-Format fields are written into this structure by the PM_tformat_receiveDataIDx() functions.

```
//
typedef struct PM_tformat_DataStruct {
    uint16_t    controlField;
    uint16_t    statusField;
    uint16_t    dataField0;
    uint16_t    dataField1;
    uint16_t    dataField2;
    uint16_t    dataField3;
    uint16_t    dataField4;
    uint16_t    dataField5;
    uint16_t    dataField6;
    uint16_t    dataField7;
    uint16_t    crcField;
    uint16_t    crcCheck;
    uint16_t    eepromAddressField;
    uint16_t    eepromWrDataField;
    uint16_t    eepromRdDataField;
    volatile bool    dataReady;
    uint16_t    fifoLevel;
} PM_tformat_DataStruct;
//
```

See The functions which populate the *tformatData* fields are:

- *PM_tformat_receiveDataID0_1_7_8_C*
- *PM_tformat_receiveDataID2*
- *PM_tformat_receiveDataID3*
- *PM_tformat_receiveDataIDD*
- *PM_tformat_receiveDataID6*

Note: For debug, set this structure to a known value before calling one of these functions. This is not required, but can be helpful since these functions do not clear out unused fields.

uint16_t **tformatRxData**[PM_TFORMAT_FIELDS_MAX]

The SPI interrupt service routine fills this array with the received data from the encoder.

- The first PM_TFORMAT_TX_FIELDS_IDx RX FIFO outputs are skipped. These were filled during the request transmission and filled with 0xFFFF.
- Each FIFO level is read, masked to include only the last 8 bits, and loaded into the corresponding tformatRxData location.

The specific PM_tformat_receiveDataIDx function then uses this array to populate the appropriate received fields in the tformatData structure.

See The functions which populate the *tformatData* fields are:

- *PM_tformat_receiveDataID0_1_7_8_C*
- *PM_tformat_receiveDataID2*
- *PM_tformat_receiveDataID3*
- *PM_tformat_receiveDataIDD*
- *PM_tformat_receiveDataID6*

group **TFORMAT_SYS_GLOBAL_STRUCTURES**
[TFORMAT_COMMAND_STRUCT_SNIPPET]

Information reporting structures used by the system example.

Variables

PM_tformat_encoderStruct **encoderData**

The position, turns and encoder ID are updated after receiving this information back from the encoder.

```
//
typedef struct PM_tformat_encoderStruct {
    uint32_t position;
    uint32_t turns;
    uint16_t id;
} PM_tformat_encoderStruct;
//
```

See

- System example function: *tformat_exCommands()*
- API function: *PM_tformat_updatePositionOrTurns()*

- API function: *PM_tformat_updateEncoderID()*

PM_tformat_errorStruct **errorData**

The variables within this structure are set to 1 if a specific error is detected by the *tformat_checkError()* function.

```
//
typedef struct PM_tformat_errorStruct {
    uint16_t idCheckError;
    uint16_t crcCheckError;
    uint16_t eepromAddressError;
    uint16_t eepromDataError;
} PM_tformat_errorStruct;

//
```

See System example functions:

- *tformat_exCommands()*
- *tformat_checkError()*

PM_tformat_commandStruct **commandData**

Global structure used to track the command data sent. This information can be used for error checking after the response is received.

```
//
typedef struct PM_tformat_commandStruct {
    uint32_t controlField;
    uint32_t eepromAddress;
    uint16_t eepromData;
} PM_tformat_commandStruct;

//
```

See System example functions:

- *tformat_testEEPROMCommands()*
- *tformat_exCommands()*
- *tformat_checkError()*

uint32_t **tformatSpiRxIsrTicker**

Global variable which increments once each time the SPI RX interrupt is taken.

3.3 System-Level Functions

This section describes functions which act at the system-level. This includes initialization of the C2000 device, start-up functions and interrupt service routine.

Many of these functions make calls to the T-Format Library which is described in *the T-Format Library Documentation*.

3.3.1 System Initialization Functions

The following functions are called to initialize the example system.

void **tformat_init** (void)

Initialize the T-Format subsystem.

void **tformat_initSPIFIFO** (uint32_t devLSPCLKFreq)

Configure the SPI peripheral.

Configures the PM_TFORMAT_SPI peripheral as required for T-Format communication

Parameters

- **devLSPCLKFreq** – SPI clock rate

Returns None.

3.3.2 System Run-time Functions

The following functions are called during run-time. They perform application specific functions make calls into the solution library.

void **tformat_exCommands** (void) __attribute__((ramfunc)) __attribute__((noreturn))

Execute read-out and reset commands.

This function is the main test for the system example. It tests the following command IDs: ID0, ID1, ID2, ID3, ID7, ID8 and IDC

For each command ID:

- Clear the tformatData structure (optional) Not required, but useful for debug.
- Wait... The idle time between commands is for demo purposes. The idle time is not tied to any specification
- Setup the SPI and CLB for the command
- Start the operation
- Wait for the dataReady flag to be set by the SPI RX ISR.
- Unpack the data from the SPI FIFO into the tformatData structure
- Check for errors using *tformat_checkError()*;
- Update the turns, position, and ID when appropriate by calling the *PM_tformat_updatePositionOrTurns()* and *PM_tformat_updateEncoderID()* library functions.

This function runs in a infinite while() loop.

See The EEPROM read and write commands are tested in the function *tformat_testEEPROMCommands()*

Returns None.

void **tformat_testEEPROMCommands** (void) __attribute__((ramfunc))

Execute EEPROM read and write commands.

Test read (ID6) and write (IDD) EEPROM commands.

- a. Write all 0's to EEPROM address in the range defined by TFORMAT_EEPROM_TEST_ADDR_START and TFORMAT_EEPROM_TEST_ADDR_END.
- b. Read out the data and check that all data read is 0x0
- c. Write a decrementing pattern: 0xFF, 0xFE, 0xFD ... to the EEPROM address range
- d. Read out the pattern and check that the data matches what was written.

After each read or write:

Check the response from the encoder against the command. The controlField (CF), eepromAddressField (ADF) and eepromDataField (EDF) received from the encoder should match the command that was sent.

Note: Some write attempts will occur when the EEPROM is busy. In this case the PM_TFORMAT_EEPROM_BUSY flag will be set and the write will be re-tried by the test.

All of the other commands are tested in the *tformat_exCommands()* function.

Returns None.

uint16_t **tformat_checkError** (uint16_t controlFieldID) __attribute__((ramfunc))

Basic error check for received fields.

Provides basic checks of the following:

- a. The received control field (CF) should match the request CF
- b. The calculated CRC for the data should match the received CRC field
- c. For EEPROM read and write commands, check the request address and data fields against the received fields.

Parameters

- **controlFieldID** – The control field (CF) of the request in the format PM_TFORMAT_CFIDx. For example PM_TFORMAT_CFID6 for command ID6.

void **tformat_spiRxISR** (void) __attribute__((interrupt("HPI"))) __attribute__((ramfunc))

Service the SPI FIFO receive interrupt.

This interrupt service routine will move the contents of the SPI RX FIFO into the tformatRxData array. Each FIFO word is masked with 0xFF to extract only the 8-bits of data corresponding to the for the specific T-Format field.

Note: This interrupt service routine is defined as a high-priority interrupt for performance reasons. This means the ISR can not be interrupted by another ISR (no nesting allowed). Refer to the [C2000 compiler guide](#) for more information.

T-FORMAT ENCODER INTERFACE LIBRARY

4.1 T-Format Library Integration

Reference the included System Example as a blueprint for integrating the library into an application. This section provides an overview of the steps required.

4.1.1 Resolve Resource Conflicts

Review the resources used by the T-Format Encoder Interface. The resource utilization is described in [TIDM-1011](#).

Resolve any resource conflicts with your specific application. Include the CLB tiles, XBARS and GPIO pins in this evaluation. In some cases, a specific resource was used due to the LaunchPad pinout. Your own application may have different hardware requirements and limitations. For example, a specific CLB tile may be able to access a GPIO through a specific XBAR.

4.1.2 Add Header Files

Include the following header files in the project source:

1. T-Format library header file: `#include "pm_tformat_include.h"`
2. System functions header file: `#include "tformat.h"`

4.1.3 Update Code Composer Studio Search Locations

CCS searches specific locations for header files and the library (.lib) files. Referring to the [directory structure](#) and update the search locations by the following steps:

1. In Code Composer Studio Project Explorer window: *Right click project name*
2. Select: *Properties -> Build -> C2000 Compiler Options -> Include Options*
3. Add the **directory path for each include file** to the compiler search path.
4. Once again, *Right click project name*
5. Select: *Properties -> Build -> C2000 Linker -> File search Path.*
6. Include the library binary in the linker options. For example: `pm_tformat_f28003x.lib`
7. Add the directory path for the lib to the library search path.

4.1.4 Add System-Level Functions

The System Example includes functions which you may wish to include in your project. Depending on your specific needs, you can copy functions or include the source files from the example by following these steps:

1. In Code Composer Studio: *Right click project name*
2. Select: *Add Files*
3. Select the file(s) to include.

4.1.5 Create Global Structures

The library requires global variables to share data between modules and with the system.

Listing 4.1: /boostxl_posmgr/f28003x/source/pm_tformat_main.c

```
//
// Globals used by the library
// - Lookup table for CRC calculations
// - tformatData structure to track key parameters.
//
PM_tformat_DataStruct tformatData;
volatile uint16_t tformatRxData[PM_TFORMAT_FIELDS_MAX];

#if defined(PM_TFORMAT_CRC_TABLE)
    uint16_t tformatCRCTable[PM_TFORMAT_CRCTABLE_SIZE];
#endif

//
```

There are additional globals that are used by this specific system example. These are not required for BiSS-C communication, but may be useful in your application. These are shown below.

Listing 4.2: /boostxl_posmgr/f28003x/source/pm_tformat_main.c

```
//
// These structures are used to display the received
// data in Code Composer Studio's Expressions
// window during development. Because the structure is
// set, but never used, the pragma RETAIN
// has been applied. Volatile is used for the
// tformatSpiRxIsrTicker since it is updated within
// the SPI ISR.
//
#pragma RETAIN (encoderData);
#pragma RETAIN (errorData);
#pragma RETAIN (commandData);

PM_tformat_encoderStruct encoderData;
PM_tformat_errorStruct errorData;
PM_tformat_commandStruct commandData;
volatile uint32_t tformatSpiRxIsrTicker;

//
```

4.1.6 Initialize the System

Refer to the example *system initialization functions*. Some functions may require modification for your specific system. Check that the TFORMAT_FREQ_DIVIDER is correctly defined for your device CLB clock frequency. This is defined in the system tformat.h header file.

4.1.7 Implement Background Loop, SPI ISR

Use the provided *system run-time functions* as a template, implement the following functions:

End-of-frame interrupt (SPI RX ISR)

Populate the RX data array with the content of the SPI RX FIFO.

Listing 4.3: /boostxl_posmgr/shared/source/tformat.c

```
//
    #pragma MUST_ITERATE(5, 12, 1)
    for (i = 0; i < tformatData.fifoLevel; i++)
    {
        tformatRxData[i] = SPI_readDataNonBlocking(PM_TFORMAT_SPI) &
        ↪ 0xFF;
    }
//
```

Send Commands and Receive Response

An example of the run-time flow is shown below:

Listing 4.4: /solutions/boostxl_posmgr/shared/source/tformat.c

```
//
// 1) Setup the command
// 2) Start the operation (transmission)
// 3) Wait for the response to be received by the SPI.
//    The SPI RX FIFO ISR will set the dataReady flag.
// 4) Unpack, or receive, the data.
// 5) System specific: check and respond to errors
// 6) Update any encoder parameters (position, turns, ID)
//
PM_tformat_setupCommandReadoutOrReset(PM_TFORMAT_ID0,
                                       PM_TFORMAT_RX_CLOCKS_ID0,
                                       PM_TFORMAT_RX_FIELDS_ID0,
                                       PM_TFORMAT_TX_CLOCKS_ID0,
                                       PM_TFORMAT_FIFO_LEVEL_ID0);

PM_tformat_startOperation();
while(tformatData.dataReady != true) {}
PM_tformat_receiveDataID0_1_7_8_C();
if(tformat_checkError(PM_TFORMAT_CFID0) != PM_TFORMAT_PASS)
{
    ESTOP0;
}
encoderData.position =
```

(continues on next page)

(continued from previous page)

```
PM_tformat_updatePositionOrTurns(tformatData.dataField0,  
                                tformatData.dataField1,  
                                tformatData.dataField2);  
  
//
```

4.2 T-Format Library Porting to CPU2

Some devices include a C28x CPU2. This section provides an overview of the steps to run the encoder interface from CPU2 instead of CPU1.

4.2.1 Resolve Resource Conflicts

If you are incorporating the library into your own project on CPU2, it is suggested to get a simple example running on CPU1 with any and all resource conflicts within your application resolved. Then focus on porting from CPU1 to CPU2. For information on incorporating the library into your own application, refer to *T-Format Library Integration*.

4.2.2 CPU1 Project

The project must be built with the pre-defined symbol CPU1.

The CPU1 sub-system initialization will include these steps:

- Initialize the device
- Allocate desired global memory blocks to CPU2
- Configure the CLB tiles for T-Format
- Allocate the peripheral access control and CPU for CLB tiles to CPU2
- Configure the PinMux, CLB_INPUT/OUTPUTXBAR, and CLB MUX as needed.
- Allocate needed GPIO pins to CPU2.
- Allocate the SPI peripheral to CPU2
- Boot CPU2
- Continue configuration of CPU1 sub-system

For F2837xD:

- The CLBx Tile is clocked by the ePWMx clock. That is, CLB1 is clocked by ePWM1 clock CLB2 by ePWM2, and so forth. This means the ePWMx module must also be assigned to CPU2 as well.

4.2.3 CPU2 Project

The project must be built with the pre-defined symbol CPU2.

The CPU2 sub-system initialization will include these steps:

- Enable the clock to the CLB tiles
- Set the T-Format frequency
- Setup the SPI interrupt in the PIE vector table, PIE IER register, and CPU IER register
- Enable the SPI peripheral

For F2837xD:

- To enable the clock to the CLB Tiles, enable the clocks to the corresponding ePWMx modules. For example: enable ePWM1 clock CPU2 will also enable the clock to the CLB1 Tile.

4.3 Global Variables

The library uses the following global variables to share receive field information.

group **TFORMAT_API_DATA_STRUCT**
[TFORMAT_API_DATA_STRUCT_SNIPPET]

Information sharing structure

Variables

PM_tformat_DataStruct **tformatData**

The received T-Format fields are written into this structure by the PM_tformat_receiveDataIDx() functions.

```
//
typedef struct PM_tformat_DataStruct {
    uint16_t    controlField;
    uint16_t    statusField;
    uint16_t    dataField0;
    uint16_t    dataField1;
    uint16_t    dataField2;
    uint16_t    dataField3;
    uint16_t    dataField4;
    uint16_t    dataField5;
    uint16_t    dataField6;
    uint16_t    dataField7;
    uint16_t    crcField;
    uint16_t    crcCheck;
    uint16_t    eepromAddressField;
    uint16_t    eepromWrDataField;
    uint16_t    eepromRdDataField;
    volatile bool    dataReady;
    uint16_t    fifoLevel;
}
```

(continues on next page)

(continued from previous page)

```

} PM_tformat_DataStruct;

//

```

See The functions which populate the *tformatData* fields are:

- *PM_tformat_receiveDataID0_1_7_8_C*
- *PM_tformat_receiveDataID2*
- *PM_tformat_receiveDataID3*
- *PM_tformat_receiveDataIDD*
- *PM_tformat_receiveDataID6*

Note: For debug, set this structure to a known value before calling one of these functions. This is not required, but can be helpful since these functions do not clear out unused fields.

uint16_t **tformatRxData**[PM_TFORMAT_FIELDS_MAX]

The SPI interrupt service routine fills this array with the received data from the encoder.

- The first PM_TFORMAT_TX_FIELDS_IDx RX FIFO outputs are skipped. These were filled during the request transmission and filled with 0xFFFF.
- Each FIFO level is read, masked to include only the last 8 bits, and loaded into the corresponding tformatRxData location.

The specific PM_tformat_receiveDataIDx function then uses this array to populate the appropriate received fields in the tformatData structure.

See The functions which populate the *tformatData* fields are:

- *PM_tformat_receiveDataID0_1_7_8_C*
- *PM_tformat_receiveDataID2*
- *PM_tformat_receiveDataID3*
- *PM_tformat_receiveDataIDD*
- *PM_tformat_receiveDataID6*

4.4 T-Format Encoder Interface API

This section describes the T-Format Encoder Interface Application Programming Interface (API). Usage of these functions are demonstrated by the system example.

4.4.1 Initialization API

The following functions initialize the system.

void **PM_tformat_generateCRCTable** (uint16_t nBits, uint16_t polynomial, uint16_t *pTable)
Initialize the CRC Lookup Table.

Generate a look-up table of 256 entries for a given CRC polynomial with specified number of bits. This table is only used if the C28x generates a CRC.

Note: This function is used if the C28x generates either the transmit or receive CRC. It is not used if the CLB generates a RX CRC or if the VCRC module generates a TX CRC. This behavior is controlled by the defines in the library header file.

Parameters

- **nBits** – number of bits of the given polynomial
- **polynomial** – polynomial used for the CRC calculations
- **pTable** – pointer to the CRC table

Returns None.

void **PM_tformat_setupPeriph** ()
Initialize the CLB tiles and SPI module.

The CLB input MUX was configured earlier in the system-level XBAR configuration.

Returns none

void **PM_tformat_setFreq** (uint32_t freqDiv)
Scale to generate T-format clock.

This function configures the CLB counters in order to generate a T-format clock at 400ns (2.5 MHz frequency).

Parameters

- **freqDiv** – Specified as TFORMAT_FREQ_DIVIDER in the tformat.h

Returns none

4.4.2 Run-time API

An example of the runtime flow is shown below:

Listing 4.5: /solutions/boostxl_posmgr/shared/source/tformat.c

```
//
// 1) Setup the command
// 2) Start the operation (transmission)
// 3) Wait for the response to be received by the SPI.
//    The SPI RX FIFO ISR will set the dataReady flag.
// 4) Unpack, or receive, the data.
// 5) System specific: check and respond to errors
// 6) Update any encoder parameters (position, turns, ID)
```

(continues on next page)

(continued from previous page)

[illegible]

Setup and Send Command

The following functions are called during run-time to setup and start a command transmission.

```
void PM_tformat_setupCommandReadoutOrReset (enum commandID0_1_2_3_7_8_C, uint16_t  
tformatRXClocks, uint16_t tformatRXFields, uint16_t  
tformatTXClocks, uint16_t tformatFIFOLevel)  
__attribute__((ramfunc))
```

Setup a Data Readout or Reset Command.

This function:

- Loads the PM_TFORMAT_SPI TX FIFO with the command field
- Configures the SPI and CLB to send the command and receive the response.
- Sets the global variable tformatData.dataReady to false.

See For read or write of EEPROM use:

- *PM_tformat_setupCommandReadEEPROM*
- *PM_tformat_setupCommandWriteEEPROM*

Note:

- This function is valid for data readout commands (ID0, 1, 2) and for reset commands (ID7, 8 and C).
- This function not valid for an EEPROM read (IDD) or EEPROM write (ID6).

Parameters

- **commandID0_1_2_3_7_8_C** – Command to be sent PM_TFORMAT_IDx

- **tformatRXClocks** – PM_TFORMAT_RX_CLOCKS_IDx SPI clocks required to send the request
- **tformatRXFields** – PM_TFORMAT_RX_FIELDS_IDx Number of fields in the response
- **tformatTXClocks** – PM_TFORMAT_TX_CLOCKS_IDx SPI clocks required to receive the response
- **tformatFIFOLevel** – PM_TFORMAT_FIFO_LEVEL_IDx FIFO level required to send the request and receive the response

Returns None.

void **PM_tformat_setupCommandReadEEPROM** (uint16_t eepromAddr) __attribute__((ramfunc))

Setup the Command for an EEPROM Read.

This function:

- Loads the PM_TFORMAT_SPI TX FIFO with the fields required to read the encoder's EEPROM.
- Configures the SPI and CLB to send the command and receive the response
- Sets the global variable tformatData.dataReady to false.

Parameters

- **eepromAddr** – Address to be read

Returns None.

void **PM_tformat_setupCommandWriteEEPROM** (uint16_t eepromAddr, uint16_t eepromData) __attribute__((ramfunc))

Setup the Command for an EEPROM Write.

This function:

- Loads the PM_TFORMAT_SPI TX FIFO with the fields required to write to the encoder's EEPROM.
- Configures the SPI and CLB to send the command and receive the response.
- Sets the global variable tformatData.dataReady to false.

Parameters

- **eepromAddr** – Address to be written to
- **eepromData** – Data to write

Returns None.

void **PM_tformat_startOperation** (void)

Start the T-Format Transaction.

This function starts the transmission of the request.

See This function is called after the appropriate setupCommand function:

- *PM_tformat_setupCommandReadEEPROM*
- *PM_tformat_setupCommandWriteEEPROM*
- *PM_tformat_setupCommand*

Returns none

Retrieve Response

The following functions retrieve and parse the encoder's response.

void **PM_tformat_receiveDataID0_1_7_8_C** (void) __attribute__((ramfunc))
Receive Data for ID0, ID1, ID7, ID8 or IDC.

Call this function when tformatData.dataReady is true indicating the data is ready in the tformatRxData array.

This function parses the tformatRxData array to populate the following tformatData structure members with the received data:

- controlField
- statusField
- dataField0, dataField1, and dataField2
- crcField

The tformatData.crcCheck variable is populated with the calculated CRC of the received data. This can be used for error checking against the received crcField.

See Other receiveData functions are:

- *PM_tformat_receiveDataID2()*
- *PM_tformat_receiveDataID3()*
- *PM_tformat_receiveDataIDD()*
- *PM_tformat_receiveDataID6()*

Note:

- This function is valid for requests ID0, 1, 7, 8 and C only.
 - For convenience, the library header file includes aliases to allow this function to be called using the format: PM_tformat_receiveDataIDx where x is 0, 1, 7, 8 or C.
 - This function does not clear unused members of the tformatData structure. This is an optional step the calling application can take if desired.
 - It is the responsibility of the calling application to perform error checking of the data.
-

Returns none

void **PM_tformat_receiveDataID2** (void) __attribute__((ramfunc))
Receive Data for ID2.

Call this function when tformatData.dataReady is true indicating the data is ready in the tformatRxData array.

This function parses the tformatRxData array to populate the following tformatData structure members with the received data:

- controlField
- statusField

- dataField0
- crcField

The tformatData.crcCheck variable is populated with the calculated CRC of the received data. This can be used for error checking against the received crcField.

See Other receiveData functions are:

- *PM_tformat_receiveDataID0_1_7_8_C()*
- *PM_tformat_receiveDataID3()*
- *PM_tformat_receiveDataIDD()*
- *PM_tformat_receiveDataID6()*

Note:

- This function is valid for requests ID2 only
 - This function does not clear unused members of the tformatData structure. This is an optional step the calling application can take if desired.
 - It is the responsibility of the calling application to perform error checking of the data.
-

Returns none

void **PM_tformat_receiveDataID3** (void) __attribute__((ramfunc))
Receive Data for ID3.

Call this function when tformatData.dataReady is true indicating the data is ready in the tformatRxData array.

This function parses the tformatRxData array to populate the following tformatData structure members with the received data:

- controlField (CF)
- statusField (SF)
- dataField0 - dataField7 (DF)
- crcField

The tformatData.crcCheck variable is populated with the calculated CRC of the received data. This can be used for error checking against the received crcField.

See Other receiveData functions are:

- *PM_tformat_receiveDataID0_1_7_8_C()*
- *PM_tformat_receiveDataID2()*
- *PM_tformat_receiveDataIDD()*
- *PM_tformat_receiveDataID6()*

Note:

- This function is valid for requests ID3 only

- This function does not clear unused members of the tformatData structure. This is an optional step the calling application can take if desired.
 - It is the responsibility of the calling application to perform error checking of the data.
-

Returns none

uint16_t **PM_tformat_receiveDataIDD** (void) __attribute__((ramfunc))
Receive data for IDD (read from EEPROM)

Call this function when tformatData.dataReady is true indicating the data is ready in the tformatRxData array.

This function parses the tformatRxData array to populate the tformatData structure members:

- controlField (CF)
- eepromAddressField (ADF)
- eepromRdDataField (EDF)
- crcField

The tformatData.crcCheck variable is populated with the calculated CRC of the received data. This can be used for error checking against the received crcField.

See Other receiveData functions are:

- *PM_tformat_receiveDataID0_1_7_8_C()*
- *PM_tformat_receiveDataID2()*
- *PM_tformat_receiveDataID3()*
- *PM_tformat_receiveDataID6()*

Note:

- This function is valid for requests IDD only
 - This function does not clear unused members of the tformatData structure. This is an optional step the calling application can take if desired.
 - It is the responsibility of the calling application to perform error checking of the data.
-

Returns indicates if the EEPROM was BUSY

- PM_TFORMAT_EEPROM_BUSY The busy bit was set in the response
- PM_TFORMAT_PASS The busy bit was not set

uint16_t **PM_tformat_receiveDataID6** (void) __attribute__((ramfunc))
Receive data for ID6 (write to EEPROM)

Call this function when tformatData.dataReady is true indicating the data is ready in the tformatRxData array.

This function parses the tformatRxData array to populate the tformatData structure members:

- controlField (CF)
- eepromAddressField (ADF)

- `eeepromWrDataField` (EDF)
- `crcField`

The `tformatData.crcCheck` variable is populated with the calculated CRC of the received data. This can be used for error checking against the received `crcField`.

See Other receiveData functions are:

- *`PM_tformat_receiveDataID0_1_7_8_C()`*
- *`PM_tformat_receiveDataID2()`*
- *`PM_tformat_receiveDataID3()`*
- *`PM_tformat_receiveDataIDD()`*

Note:

- This function is valid for requests ID6 only
 - This function does not clear unused members of the `tformatData` structure. This is an optional step the calling application can take if desired.
 - It is the responsibility of the calling application to perform error checking of the data.
-

Returns

- `PM_TFORMAT_EEPROM_BUSY` The busy bit was set in the response
- `PM_TFORMAT_PASS` The busy bit was not set

Update Encoder Parameters

The following functions used to update the `encoderData` parameters.

`uint32_t PM_tformat_updatePositionOrTurns` (`uint16_t field0`, `uint16_t field1`, `uint16_t field2`)
 `__attribute__((ramfunc))`

Update the position or turns.

Inverts the received bit sequence for the actual position and turns data Refer to the T-format specification for which command fields contain position or turn information.

Parameters

- **field0** – first field received
- **field1** – second field received
- **field2** – third field received

Returns 32-bit position or turns

`uint16_t PM_tformat_updateEncoderID` (`uint16_t enidField`) `__attribute__((ramfunc))`

Update the encoder ID.

Inverts the received bit sequence of ENID to form the actual encoder ID Refer to the T-format specification for which command fields contain encoder ID (ENID) information.

Parameters

- **enidField** – ENID field received from the encoder

Returns 16-bit encoderID

4.5 Library Private Functions

The following functions are not called directly by the application. They are called from other functions within the library.

4.5.1 Initialization Functions

void **tformat_initCLBXBAR** (void)

Configure the CLB XBAR.

Configures the CLB XBAR. This function is generated by SysConfig

Returns None.

4.5.2 Run-time Configuration Functions

static inline void **tformat_configureCLBLen** (uint16_t transmitClocks, uint16_t receiveClocks)
__attribute__((ramfunc))

Configure the CLB.

Configures the CLB as required for the specific command.

Returns None.

static inline void **tformat_configureSPILen** (void) __attribute__((ramfunc))

Configure the SPI FIFO depth.

Configures the PM_TFORMAT_SPI FIFO depth as required for the specific command.

Uses the global parameter tformatData.fifoLevel to set both the TX and the RX SPI FIFO level.

Returns None.

static inline void **tformat_resetCLB** (uint16_t tformatRXFields) __attribute__((ramfunc))

Reset the CLB.

Reset portions of the CLB. Configure the CLB for the next transaction with the encoder

Parameters

- **tformatRXFields** – The number of fields in the expected response this is in the form PM_TFORMAT_RX_FIELDS_IDx where x is the ID number of the command

Returns None.

4.5.3 CRC Run-time Functions

Which CRC generation method is used depends on the library configuration.

Refer to [the configuration section](#) for more information.

Listing 4.6: /libraries/position_sensing/tformat/include/pm_tformat_include.h

```
//
// Define how the CRC is generated. Supported options are:
//
// #define PM_TFORMAT_RX_CRC_BY_C28X
// #define PM_TFORMAT_TX_CRC_BY_C28X
//     The CRC for the data (TX or RX) is generated by the C28x using a
//     lookup table. This method takes the most C28x bandwidth and
//     memory.
//
// #define PM_TFORMAT_RX_CRC_BY_CLB
//     The received data CRC is generated on-the-fly by the CLB as the
//     data is received.
//     This requires 1 CLB tile.
//     This option requires CLB type 2 or later.
//
// #define PM_TFORMAT_TX_CRC_BY_VCRC
//     The transmit data CRC is generated by the VCRC extension to the C28x.
//     This option requires that the device has the VCRC extension.
//
```

uint16_t tformat_getCRCID0_1_7_8_C (void) __attribute__((ramfunc))

Calculate the return data CRC for a specific command ID.

This function creates a data packet from the tformatData structure. The data fields used are specific to the command ID identified in the name of the function. For example [tformat_getCRCID2\(\)](#) will use response data fields specific to command ID2.

Depending on the device, different features may be used to generate actual the CRC value. This function calls the appropriate CRC routine based on the library's configuration.

See The getCRCIDx functions are:

- [tformat_getCRCID0_1_7_8_C](#) Valid for ID0, ID1, ID7, ID8 and IDC
- [tformat_getCRCID2](#) Valid for ID2
- [tformat_getCRCID3](#) Valid for ID3
- [tformat_getCRCIDD](#) Valid for IDD
- [tformat_getCRCID6](#) Valid for ID6

Returns crcResult 8-bit calculated CRC value

uint16_t tformat_getCRCID2 (void) __attribute__((ramfunc))

Calculate the return data CRC for a specific command ID.

See [tformat_getCRCID0_1_7_8_C](#)

uint16_t **tformat_getCRCID3** (void) __attribute__((ramfunc))

Calculate the return data CRC for a specific command ID.

See *tformat_getCRCID0_1_7_8_C*

uint16_t **tformat_getCRCIDD** (void) __attribute__((ramfunc))

Calculate the return data CRC for a specific command ID.

See *tformat_getCRCID0_1_7_8_C*

uint16_t **tformat_getCRCID6** (void) __attribute__((ramfunc))

Calculate the return data CRC for a specific command ID.

See *tformat_getCRCID0_1_7_8_C*

CLB CRC

uint16_t **tformat_getRxCRCbyCLB** () __attribute__((ramfunc))

Get the CRC for a received response.

Get the CRC of the encoder's response. The CRC calculation is performed while the response is received.

To use this function, set

```
#define PM_TFORMAT_RX_CRC_BY_CLB 1
```

in the library header file.

Note: Requires CLB type 2 or later and one CLB tile. It is assumed the CLB tile has been properly configured during system initialization.

Returns CRC 8-bit calculated CRC

VCRC CRC

uint16_t **tformat_getTxCRCbyVCRC** (uint16_t *msgBuffer)

Calculate T-Format transmit data CRC using the VCRC.

Calculate the CRC for a T-format transmit operation. The polynomial, polynomial size and data size are hard-coded for the T-format protocol.

Note: The C28x VCRC extension calculates the CRC. VCRC is not available on all devices. Refer to the device-specific data sheet.

Warning: This function assumes it will not interrupt another VCRC calculation. That is, this function does not context save the VCRC, VCRCPOLY and VCRCSIZE registers upon entry. If this not the case, the user must add the context save and restore.

Parameters

- **msgBuffer** – Pointer to the message buffer

Returns crcResult 8-bit calculated CRC value

C28x Look-up Table CRC

uint16_t **tformat_getCRC** (uint16_t nBitsData, uint16_t *msg, uint16_t *crcTable, uint16_t rxLen)
 __attribute__((ramfunc))

Calculate the CRC for the given data.

Calculate the n-bit CRC of a message buffer by using a lookup table to get the CRC of each byte. The CRC lookup table should be initialized by calling *PM_tformat_generateCRCTable()* before this function.

Attention

- This CRC assumes that the number of data bits is always divisible by 8. This is true for T-format data.

This function can be used for both transmit CRC calculations (EEPROM read/write commands) and verification of the received CRC.

See *PM_tformat_generateCRCTable*

See This function is called by the specific tformat_getCRCIDx functions:

- *tformat_getCRCID0_1_7_8_C*
- *tformat_getCRCID2*
- *tformat_getCRCID3*
- *tformat_getCRCID6*
- *tformat_getCRCIDD*

Parameters

- **nBitsData** – number of bits of data for which the CRC is calculated
- **msg** – pointer to the data on which the CRC will be calculated
- **crcTable** – pointer to the table where the CRC values are stored
- **rxLen** – number of bytes of data for the CRC calculation

4.6 Protocol Defines

The library header file includes a number of T-Format protocol-based defines. Some of these are shown below for reference. Information on specific fields and commands can be found in the [glossery](#).

4.6.1 Command and Response Format

Listing 4.7: /libraries/position_sensing/tformat/include/pm_tformat_include.h

```
//
// Transmit packet (request)
//   Number of 10-bit fields (command) for each Data ID code:
//
//   ID0: TX 1 field      CF
//   ID1: TX 1 field      CF
//   ID2: TX 1 field      CF
//   ID3: TX 1 field      CF
//   ID7: TX 1 field      CF
//   ID8: TX 1 field      CF
//   IDC: TX 1 field      CF
//   ID6: TX 4 fields     CF + ADF + EDF + CRC      (EEPROM write)
//   IDD: TX 3 fields     CF + ADF + CRC           (EEPROM read)
//
// Receive packet (response)
//   Number of 10-bit fields received for each request
//
//   ID0: RX 6 fields:   CF + SF + DF0 + DF1 + DF2 + CRC
//                       DF0-DF2 == ABS0-ABS2
//
//   ID1: RX 6 fields:   CF + SF + DF0 + DF1 + DF2 + CRC
//                       DF0-DF2 == ABM0-ABM2
//
//   ID2: RX 4 fields:   CF + SF + DF0 + CRC
//                       DF0 == ENID
//
//   ID3: RX 11 fields:  CF + SF + DF0 + DF1 + DF2....DF7 + CRC
//                       DF0-DF2 == ABS0-ABS2
//                       DF3 == ENID
//                       DF4-DF6 == ABM0-ABM2
//                       DF7 == ALMC
//
//   ID7: RX 6 fields:   CF + SF + DF0 + DF1 + DF2 + CRC
//                       DF0-DF2 == ABS0-ABS2
//
//   ID8: RX 6 fields:   CF + SF + DF0 + DF1 + DF2 + CRC
//                       DF0-DF2 == ABS0-ABS2
//
//   IDC: RX 6 fields:   CF + SF + DF0 + DF1 + DF2 + CRC
//                       DF0-DF2 == ABS0-ABS2
//
//   ID6: RX 4 fields:   CF + ADF + EDF + CRC
//
```

(continues on next page)

(continued from previous page)

```

//      IDD: RX 4 fields:  CF + ADF + EDF + CRC
//
//
// *****
#define PM_TFORMAT_TX_FIELDS_ID0      1u
#define PM_TFORMAT_TX_FIELDS_ID1      1u
#define PM_TFORMAT_TX_FIELDS_ID2      1u
#define PM_TFORMAT_TX_FIELDS_ID3      1u
#define PM_TFORMAT_TX_FIELDS_ID7      1u
#define PM_TFORMAT_TX_FIELDS_ID8      1u
#define PM_TFORMAT_TX_FIELDS_IDC      1u
#define PM_TFORMAT_TX_FIELDS_ID6      4u
#define PM_TFORMAT_TX_FIELDS_IDD      3u

#define PM_TFORMAT_RX_FIELDS_ID0      6u
#define PM_TFORMAT_RX_FIELDS_ID1      6u
#define PM_TFORMAT_RX_FIELDS_ID2      4u
#define PM_TFORMAT_RX_FIELDS_ID3      11u
#define PM_TFORMAT_RX_FIELDS_ID7      6u
#define PM_TFORMAT_RX_FIELDS_ID8      6u
#define PM_TFORMAT_RX_FIELDS_IDC      6u
#define PM_TFORMAT_RX_FIELDS_ID6      4u
#define PM_TFORMAT_RX_FIELDS_IDD      4u

#define PM_TFORMAT_RX_FIELDS_MAX      11u
#define PM_TFORMAT_TX_FIELDS_MAX      4u
#define PM_TFORMAT_FIELDS_MAX

    \
    (PM_TFORMAT_TX_FIELDS_ID3 + PM_TFORMAT_RX_FIELDS_ID3)

//

```

4.6.2 SPI Clocks Required to Send/receive

Listing 4.8: /libraries/position_sensing/tformat/include/pm_tformat_include.h

```

//
//
// *****
// Number of SPI clocks required to transmit a request is:
// = 1 idle bit + (number of TX fields * number of field bits)
// = 1 idle bit + total number of TX bits
//
//
// *****
#define PM_TFORMAT_TX_CLOCKS_ID0 (PM_TFORMAT_IDLE_BIT
    \
    + (PM_TFORMAT_TX_FIELDS_ID0 * PM_TFORMAT_FIELD_BITS))
#define PM_TFORMAT_TX_CLOCKS_ID1 (PM_TFORMAT_IDLE_BIT
    \

```

(continues on next page)

(continued from previous page)

```

        + (PM_TFORMAT_TX_FIELDS_ID1 * PM_TFORMAT_FIELD_BITS) )
#define PM_TFORMAT_TX_CLOCKS_ID2 (PM_TFORMAT_IDLE_BIT
→      \
        + (PM_TFORMAT_TX_FIELDS_ID2 * PM_TFORMAT_FIELD_BITS) )
#define PM_TFORMAT_TX_CLOCKS_ID3 (PM_TFORMAT_IDLE_BIT
→      \
        + (PM_TFORMAT_TX_FIELDS_ID3 * PM_TFORMAT_FIELD_
→BITS))
#define PM_TFORMAT_TX_CLOCKS_ID6 (PM_TFORMAT_IDLE_BIT
→      \
        + (PM_TFORMAT_TX_FIELDS_ID6 * PM_TFORMAT_FIELD_BITS) )
#define PM_TFORMAT_TX_CLOCKS_ID7 (PM_TFORMAT_IDLE_BIT
→      \
        + (PM_TFORMAT_TX_FIELDS_ID7 * PM_TFORMAT_FIELD_BITS) )
#define PM_TFORMAT_TX_CLOCKS_ID8 (PM_TFORMAT_IDLE_BIT
→      \
        + (PM_TFORMAT_TX_FIELDS_ID8 * PM_TFORMAT_FIELD_BITS) )
#define PM_TFORMAT_TX_CLOCKS_IDC (PM_TFORMAT_IDLE_BIT
→      \
        + (PM_TFORMAT_TX_FIELDS_IDC * PM_TFORMAT_FIELD_BITS) )
#define PM_TFORMAT_TX_CLOCKS_IDD (PM_TFORMAT_IDLE_BIT
→      \
        + (PM_TFORMAT_TX_FIELDS_IDD * PM_TFORMAT_FIELD_BITS) )

//
→*****
//
// Number of SPI clocks required to receive a response is:
// = number of RX fields * number of field bits
// = total number of RX bits
//
// In the case of an error, the encoder will respond with ID3. An
→error
// can be the wrong TX CRC or an invalid command. Therefore the RX
→clocks
// will correspond to an ID3 response or PM_TFORMAT_TX_FIELDS_MAX
//
//
→*****

#define PM_TFORMAT_RX_CLOCKS_ID0
→      \
        (PM_TFORMAT_RX_FIELDS_ID0 * PM_TFORMAT_FIELD_
→BITS)
#define PM_TFORMAT_RX_CLOCKS_ID1
→      \
        (PM_TFORMAT_RX_FIELDS_ID1 * PM_TFORMAT_FIELD_
→BITS)
#define PM_TFORMAT_RX_CLOCKS_ID2
→      \
        (PM_TFORMAT_RX_FIELDS_ID2 * PM_TFORMAT_FIELD_
→BITS)

```

(continues on next page)

```
#define PM_TFORMAT_RX_CLOCKS_ID3
    ↪
        (PM_TFORMAT_RX_FIELDS_ID3 * PM_TFORMAT_FIELD_
    ↪BITS)
#define PM_TFORMAT_RX_CLOCKS_ID6
    ↪
        (PM_TFORMAT_RX_FIELDS_ID6 * PM_TFORMAT_FIELD_
    ↪BITS)
#define PM_TFORMAT_RX_CLOCKS_ID7
    ↪
        (PM_TFORMAT_RX_FIELDS_ID7 * PM_TFORMAT_FIELD_
    ↪BITS)
#define PM_TFORMAT_RX_CLOCKS_ID8
    ↪
        (PM_TFORMAT_RX_FIELDS_ID8 * PM_TFORMAT_FIELD_
    ↪BITS)
#define PM_TFORMAT_RX_CLOCKS_IDC
    ↪
        (PM_TFORMAT_RX_FIELDS_IDC * PM_TFORMAT_FIELD_
    ↪BITS)
#define PM_TFORMAT_RX_CLOCKS_IDD
    ↪
        (PM_TFORMAT_RX_FIELDS_IDD * PM_TFORMAT_FIELD_
    ↪BITS)
#define PM_TFORMAT_RX_CLOCKS_MAX
    ↪
        (PM_TFORMAT_RX_FIELDS_MAX * PM_TFORMAT_FIELD_
    ↪BITS)
#define PM_TFORMAT_RX_CLOCKS_IDX
    ↪
        (PM_TFORMAT_RX_FIELDS_ID3 * PM_TFORMAT_FIELD_
    ↪BITS)

//
```

```
//
//
↳ *****
//
// Since SPISTEN is always active, the RX FIFO will see both the
// TX and RX data. Thus the required FIFO level is the sum of the_
↳ two.
// In the current design, this FIFO level is used for both TX and RX.
//
// On some devices the CLB can directly control the SPISTEN signal. _
↳ Changing
```

(continued from previous page)

```

// the state of the signal is a potential design change that a
→developer may
// want to consider.
//
//
→*****
#define PM_TFORMAT_FIFO_LEVEL_ID0                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_ID0 + PM_TFORMAT_TX_FIELDS_
→ID0)
#define PM_TFORMAT_FIFO_LEVEL_ID1                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_ID1 + PM_TFORMAT_TX_FIELDS_
→ID1)
#define PM_TFORMAT_FIFO_LEVEL_ID2                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_ID2 + PM_TFORMAT_TX_FIELDS_
→ID2)
#define PM_TFORMAT_FIFO_LEVEL_ID3                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_ID3 + PM_TFORMAT_TX_FIELDS_
→ID3)
#define PM_TFORMAT_FIFO_LEVEL_ID6                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_ID6 + PM_TFORMAT_TX_FIELDS_
→ID6)
#define PM_TFORMAT_FIFO_LEVEL_ID7                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_ID7 + PM_TFORMAT_TX_FIELDS_
→ID7)
#define PM_TFORMAT_FIFO_LEVEL_ID8                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_ID8 + PM_TFORMAT_TX_FIELDS_
→ID8)
#define PM_TFORMAT_FIFO_LEVEL_IDC                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_IDC + PM_TFORMAT_TX_FIELDS_
→IDC)
#define PM_TFORMAT_FIFO_LEVEL_IDD                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_IDD + PM_TFORMAT_TX_FIELDS_
→IDD)
#define PM_TFORMAT_FIFO_LEVEL_MAX                                ↵
→      \
      (PM_TFORMAT_RX_FIELDS_MAX + PM_TFORMAT_TX_FIELDS_
→MAX)
//

```

4.6.4 Data ID Code Format

Listing 4.10: /libraries/position_sensing/tformat/include/pm_tformat_include.h

```
//
//
// *****
//
// Data ID Code
// Unique code for each command. Defined in the T-Format_
// specification.
// IDx = code (4-bits) + parity (1-bit)
//
//
// *****
#define PM_TFORMAT_CODEID0 0x00u // 0b0000:0 = code0:parity
#define PM_TFORMAT_CODEID1 0x11u // 0b1000:1 = code1:parity
#define PM_TFORMAT_CODEID2 0x09u // 0b0100:1 = code2:parity
#define PM_TFORMAT_CODEID3 0x18u // 0b1100:0 = code3:parity
#define PM_TFORMAT_CODEID6 0x0Cu // 0b0110:0 = code6:parity
#define PM_TFORMAT_CODEIDD 0x17u // 0b1011:1 = codeD:parity
#define PM_TFORMAT_CODEID7 0x1Du // 0b1110:1 = code7:parity
#define PM_TFORMAT_CODEID8 0x03u // 0b0001:1 = code8:parity
#define PM_TFORMAT_CODEIDC 0x06u // 0b0011:0 = codeC:parity
//
```

4.6.5 Command Field (CF)

Listing 4.11: /libraries/position_sensing/tformat/include/pm_tformat_include.h

```
//
//
// *****
//
// Control Field (CF)
// First 10-bit field in any transmitted or received packet
//
// idle start sink code Data ID code Delimiter
// ...1,1 0 010 xxxx:x 1
//
// * 1-bit start: fixed (0)
// * 3-bit sink code: fixed (0,1,0)
// * 5-bit DataIDCode: code (4-bits) + parity (1-bit)
// * 1-bit Delimiter: fixed (1)
//
// Note: The delimiter is added in the PM_TFORMAT_IDx enumerations
//
//
// *****
#define PM_TFORMAT_CFID0 (PM_TFORMAT_CODEID0 | 0x40u) //CF:
//0b010:0000:0 = 0x40 (continues on next page)
```

(continued from previous page)

```

#define PM_TFORMAT_CFID1 (PM_TFORMAT_CODEID1 | 0x40u) //CF:
    ↳0b010:1000:1 = 0x51
#define PM_TFORMAT_CFID2 (PM_TFORMAT_CODEID2 | 0x40u) //CF:
    ↳0b010:0100:1 = 0x49
#define PM_TFORMAT_CFID3 (PM_TFORMAT_CODEID3 | 0x40u) //CF:
    ↳0b010:1100:0 = 0x58
#define PM_TFORMAT_CFID6 (PM_TFORMAT_CODEID6 | 0x40u) //CF:
    ↳0b010:0110:0 = 0x4C
#define PM_TFORMAT_CFIDD (PM_TFORMAT_CODEIDD | 0x40u) //CF:
    ↳0b010:1011:1 = 0x57
#define PM_TFORMAT_CFID7 (PM_TFORMAT_CODEID7 | 0x40u) //CF:
    ↳0b010:1110:1 = 0x5D
#define PM_TFORMAT_CFID8 (PM_TFORMAT_CODEID8 | 0x40u) //CF:
    ↳0b010:0001:1 = 0x43
#define PM_TFORMAT_CFIDC (PM_TFORMAT_CODEIDC | 0x40u) //CF:
    ↳0b010:0011:0 = 0x46

//

```

4.6.6 CRC Calculations

Listing 4.12: /libraries/position_sensing/tformat/include/pm_tformat_include.h

```

//
//
    ↳*****
//
// CRC related declarations
// T-format polynomial: x^8 + 1
//
//
    ↳*****
#define PM_TFORMAT_CRC_START          0
#define PM_TFORMAT_NBITS_POLY        8u
#define PM_TFORMAT_POLY              0x01u
#define PM_TFORMAT_CRCTABLE_SIZE     256u
#define PM_TFORMAT_CRC_MASK          0xFFu

//
    ↳*****
//
// Number of bits to use in CRC calculations:
//
// Each field is a total of 10 bits, with 8 bits being actual data.
// This defines the number of bytes, or bits, of data used to
    ↳calculate the CRC.
// The number of bytes used is one less than the total bytes in the
    ↳packet.
// The extra byte received is the CRC sent from the encoder.

```

(continues on next page)

(continued from previous page)

```

//
// For example:
//      ID7: RX 6 fields:  CF + SF + DF0 + DF1 + DF2 + CRC
//                        -----
//                        5 fields used to calc CRC
//
//
//
→ *****
#define PM_TFORMAT_RX_CRC_BYTES_ID0    (PM_TFORMAT_RX_FIELDS_ID0 - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_ID1    (PM_TFORMAT_RX_FIELDS_ID1 - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_ID2    (PM_TFORMAT_RX_FIELDS_ID2 - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_ID3    (PM_TFORMAT_RX_FIELDS_ID3 - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_ID6    (PM_TFORMAT_RX_FIELDS_ID6 - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_IDD    (PM_TFORMAT_RX_FIELDS_IDD - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_ID7    (PM_TFORMAT_RX_FIELDS_ID7 - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_ID8    (PM_TFORMAT_RX_FIELDS_ID8 - 1u)
#define PM_TFORMAT_RX_CRC_BYTES_IDC    (PM_TFORMAT_RX_FIELDS_IDC - 1u)

#define PM_TFORMAT_RX_CRC_BITS_ID0      (PM_TFORMAT_RX_CRC_BYTES_ID0 *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_ID1      (PM_TFORMAT_RX_CRC_BYTES_ID1 *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_ID2      (PM_TFORMAT_RX_CRC_BYTES_ID2 *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_ID3      (PM_TFORMAT_RX_CRC_BYTES_ID3 *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_ID6      (PM_TFORMAT_RX_CRC_BYTES_ID6 *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_IDD      (PM_TFORMAT_RX_CRC_BYTES_IDD *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_ID7      (PM_TFORMAT_RX_CRC_BYTES_ID7 *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_ID8      (PM_TFORMAT_RX_CRC_BYTES_ID8 *
→      \
→      PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_RX_CRC_BITS_IDC      (PM_TFORMAT_RX_CRC_BYTES_IDC *
→      \
→

```

(continues on next page)

(continued from previous page)

```

                                PM_TFORMAT_FIELD_DATA_
→BITS)

//
// *****
//
// The number of transmitted bytes, or bits, of data used to
→calculate the CRC
// for the EEPROM read/write commands. Other commands do not have a
→transmit
// CRC.
//
//
→ *****
#define PM_TFORMAT_TX_CRC_BYTES_ID6 (PM_TFORMAT_TX_FIELDS_ID6 - 1u)
#define PM_TFORMAT_TX_CRC_BYTES_IDD (PM_TFORMAT_TX_FIELDS_IDD - 1u)

#define PM_TFORMAT_TX_CRC_BITS_ID6 (PM_TFORMAT_TX_CRC_BYTES_ID6 *
→
                                PM_TFORMAT_FIELD_DATA_
→BITS)
#define PM_TFORMAT_TX_CRC_BITS_IDD (PM_TFORMAT_TX_CRC_BYTES_IDD *
→
                                PM_TFORMAT_FIELD_DATA_
→BITS)

//
// Error codes for certain T-format functions.
// Note: 16-bit error codes are used so they will be different from
→any valid
// 8-bit CRC value.
//
#define PM_TFORMAT_PASS 0
#define PM_TFORMAT_CRC_CLB_ERROR 0xFFFF
#define PM_TFORMAT_CRC_CHECK_ERROR 0xFFFFEP
#define PM_TFORMAT_EEPROM_BUSY 0xFFFFD
#define PM_TFORMAT_ID_CHECK_ERROR 0xFFFFC

//

```

4.7 Library Deprecated Functions

The following functions are deprecated and no longer used.

uint16_t **PM_tformat_setupCommand** (uint16_t dataID, uint16_t eepromAddr, uint16_t eepromData, uint16_t crc) __attribute__((deprecated("Replace with a command-specific function.")))

Setup a command (Deprecated)

To improve performance, this function has been split into command-specific functions shown below.

See Command specific functions:

- *PM_tformat_setupCommandReadEEPROM()*
- *PM_tformat_setupCommandWriteEEPROM()*
- *PM_tformat_setupCommandReadoutOrReset()*

uint16_t **PM_tformat_receiveData** (uint16_t dataID) __attribute__((deprecated("Replace with a command-specific function.")))

Receive data (Deprecated)

To improve performance, this function has been split into command-specific functions.

See Command-specific functions:

- *PM_tformat_receiveDataID0_1_7_8_C()*
- *PM_tformat_receiveDataID2()*
- *PM_tformat_receiveDataID3()*
- *PM_tformat_receiveDataIDD()*
- *PM_tformat_receiveDataID6()*

ADDITIONAL RESOURCES

5.1 Abbreviations and Acronyms

ABSx ABS0:ABS1:ABS2 is the absolute position data for 1 revolution.

ABMx ABM0:ABM1:ABM2 is the absolute multi-turn data.

ADF Address data field. Used in the read/write of EEPROM memory commands. The address has the format shown below. For further detail, refer to the T-Format specification.

	bit1						bit10
idle	start	add0:add1:add2:add3:add4:add5:add6	BUSY				delimiter
1	0	Address (LSB first)	status				1

ALMC Encoder error field. Refer to the T-format spec for details.

CF Control field. The 10-bit control field is the 1st field in any command or response frame. The format of the field is shown below. For specifics, refer to the T-Format specification.

	bit1					bit10
idle	start	sink code	cc0:cc1:cc2:cc3:cc4			delimiter
1	0	0 1 0	Data ID Code			1

CLB Configurable Logic Block

Command See Request

CRC Cyclic Redundancy Check. The polynomial used is $X^8 + 1$. The CRC field is the last field received in a response from the encoder. For EEPROM read/write commands, a CRC the last field in the transmission.

	bit1					bit10
idle	start	rc0:rc1:rc2:rc3:rc4:rc5:rc6:rc7				delimiter
1	0	CRC (LSB first)				1

Data ID Code 4-bit code plus parity that identifies a specific T-format command.

Delimiter 1-bit, always 1, at the end of each field

DFx Data Field within the response. The number of DFs within a response depends on the command sent. ID3 has the most number of data fields (DF0...DF7). The structure of a single data field is shown below:

	bit1					bit10
idle	start	dn0:dn1:dn2:dn3:dn4:dn5:dn6:dn7				delimiter
1	0	data (LSB first)				1

The meaning of the data depends on the specific command and the specific field as shown below.

ID	DF0	DF1	DF2	DF3	DF4	DF5	DF6	DF7
0	ABS0	ABS1	ABS2					
1	ABM0	ABM1	ABM2					
2	ENID							
3	ABS0	ABS1	ABS2	ENID	ABM0	ABM1	ABM2	ALMC
7	ABS0	ABS1	ABS2					
8	ABS0	ABS1	ABS2					
C	ABS0	ABS1	ABS2					

EDF EEPROM data field. Used in the EEPROM read/write commands.

ENID Encoder ID

Field A T-Format transmission, or received data, can be broken down into 10-bit fields. Each field begins with a start bit, and ends with a delimiter. The 8-bits between are data.

Request (or command) A request consists of at least a control field (CF). For EEPROM read/write requests additional fields for address, data and CRC are also transmitted.

- EEPROM read request: CF + ADF + CRC
- EEPROM write request: CF + ADF + EDF + CRC
- Other request: CF

PM Position Manager. Foundation hardware and software on C28x devices for position sensing

PM_tformat or PM_TFORMAT Prefix used for the library functions and #defines

Response The transmission data from the absolute encoder. The general format of a response is shown below. The data fields (number and meaning) in the response depend on the specific command transmitted. Refer to the DFx definition.

- EEPROM read response: CF + ADF + EDF + CRC
- EEPROM write response: CF + ADF + CRC
- Other command response: CF + SF + DF0.....DF7 + CRC

Sink Code Fixed pattern of 0,1,0. Immediately after the start bit of a control field (CF)

SPI Serial Peripheral Interface

SF Status field. The 10-bit status field is the 2nd field in any response from the encoder. The format of the field is shown below. For specifics, refer to the T-Format specification.

	bit1				bit10
idle	start	Information	error	alarm	delimiter
1	0	dd0:dd1:dd2:dd3	ea0:ea1	ca0:ca1	1

Start 1-bit, always 0, at the start of any T-format field.

T-Format For a detailed description of the specification and terminology, refer to the [T-Format Encoder Interface by Tamagawa Seiki](#) documentation

CHANGELOG

6.1 C2000 MotorControl SDK Version 5.00.00.00

- Added support for F28P65x LaunchPad
- Enabled the pull-up on the SPI SIMO (PICO) pin. This is to keep the signal high if the encoder should become disconnected.
- Updated the resetCLB function to perform a soft reset of the module.
- Fixed the argument to the SPI enable function call.

6.2 C2000 MotorControl SDK Version 4.01.00.00

Changes were focused on performance improvement, ease of use, and supporting new device features. Refer to the section *library migration* for specific details.

IMPORTANT NOTICE

Texas Instruments Incorporated and its subsidiaries (TI) reserve the right to make corrections, enhancements, improvements and other changes to its semiconductor products and services per JESD46, latest issue, and to discontinue any product or service per JESD48, latest issue. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. All semiconductor products (also referred to herein as “components”) are sold subject to TI’s terms and conditions of sale supplied at the time of order acknowledgment.

TI warrants performance of its components to the specifications applicable at the time of sale, in accordance with the warranty in TI’s terms and conditions of sale of semiconductor products. Testing and other quality control techniques are used to the extent TI deems necessary to support this warranty. Except where mandated by applicable law, testing of all parameters of each component is not necessarily performed.

TI assumes no liability for applications assistance or the design of Buyers’ products. Buyers are responsible for their products and applications using TI components. To minimize the risks associated with Buyers’ products and applications, Buyers should provide adequate design and operating safeguards.

TI does not warrant or represent that any license, either express or implied, is granted under any patent right, copyright, mask work right, or other intellectual property right relating to any combination, machine, or process in which TI components or services are used. Information published by TI regarding third-party products or services does not constitute a license to use such products or services or a warranty or endorsement thereof. Use of such information may require a license from a third party under the patents or other intellectual property of the third party, or a license from TI under the patents or other intellectual property of TI.

Reproduction of significant portions of TI information in TI data books or data sheets is permissible only if reproduction is without alteration and is accompanied by all associated warranties, conditions, limitations, and notices. TI is not responsible or liable for such altered documentation. Information of third parties may be subject to additional restrictions.

Resale of TI components or services with statements different from or beyond the parameters stated by TI for that component or service voids all express and any implied warranties for the associated TI component or service and is an unfair and deceptive business practice. TI is not responsible or liable for any such statements.

Buyer acknowledges and agrees that it is solely responsible for compliance with all legal, regulatory and safety-related requirements concerning its products, and any use of TI components in its applications, notwithstanding any applications-related information or support that may be provided by TI. Buyer represents and agrees that it has all the necessary expertise to create and implement safeguards which anticipate dangerous consequences of failures, monitor failures and their consequences, lessen the likelihood of failures that might cause harm and take appropriate remedial actions. Buyer will fully indemnify TI and its representatives against any damages arising out of the use of any TI components in safety-critical applications.

In some cases, TI components may be promoted specifically to facilitate safety-related applications. With such components, TI’s goal is to help enable customers to design and create their own end-product solutions that meet applicable functional safety standards and requirements. Nonetheless, such components are subject to these terms.

No TI components are authorized for use in FDA Class III (or similar life-critical medical equipment) unless authorized officers of the parties have executed a special agreement specifically governing such use.

Only those TI components which TI has specifically designated as military grade or “enhanced plastic” are designed and intended for use in military/aerospace applications or environments. Buyer acknowledges and agrees that any military or aerospace use of TI components which have not been so designated is solely at the Buyer’s risk, and that Buyer is solely responsible for compliance with all legal and regulatory requirements in connection with such use.

TI has specifically designated certain components which meet ISO/TS16949 requirements, mainly for automotive use. Components which have not been so designated are neither designed nor intended for automotive use; and TI will not be responsible for any failure of such components to meet such requirements.

DISCLAIMER

ALL CONTENT AND MATERIALS ON THIS SITE ARE PROVIDED “AS IS”. TI AND ITS RESPECTIVE SUPPLIERS MAKE NO REPRESENTATIONS ABOUT THE SUITABILITY OF THESE MATERIALS FOR ANY PURPOSE AND DISCLAIM ALL WARRANTIES AND CONDITIONS WITH REGARD TO THESE MATERIALS, INCLUDING BUT NOT LIMITED TO, ALL IMPLIED WARRANTIES AND CONDITIONS OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT OF ANY THIRD PARTY INTELLECTUAL PROPERTY RIGHT. NO LICENSE, EITHER EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, IS GRANTED BY TI. USE OF THE INFORMATION ON THIS SITE MAY REQUIRE A LICENSE FROM A THIRD PARTY, OR A LICENSE FROM TI.

Content on this site may contain or be subject to specific guidelines or limitations on use. All postings and use of the content on this site are subject to the Terms of Use of the site; third parties using this content agree to abide by any limitations or guidelines and to comply with the Terms of Use of this site. TI and its suppliers reserve the right to make corrections, deletions, modifications, enhancements, improvements and other changes to the content and materials, its products, programs and services at any time or to move or discontinue any content, products, programs, or services without notice.

C

commandData (*C var*), 21

E

encoderData (*C var*), 20

errorData (*C var*), 21

P

PM_tformat_generateCRCTable (*C function*), 30

PM_tformat_receiveData (*C function*), 50

PM_tformat_receiveDataID0_1_7_8_C (*C function*), 33

PM_tformat_receiveDataID2 (*C function*), 33

PM_tformat_receiveDataID3 (*C function*), 34

PM_tformat_receiveDataID6 (*C function*), 35

PM_tformat_receiveDataIDD (*C function*), 35

PM_tformat_setFreq (*C function*), 30

PM_tformat_setupCommand (*C function*), 49

PM_tformat_setupCommandReadEEPROM (*C function*), 32

PM_tformat_setupCommandReadoutOrReset (*C function*), 31

PM_tformat_setupCommandWriteEEPROM (*C function*), 32

PM_tformat_setupPeriph (*C function*), 30

PM_tformat_startOperation (*C function*), 32

PM_tformat_updateEncoderID (*C function*), 36

PM_tformat_updatePositionOrTurns (*C function*), 36

T

tformat_checkError (*C function*), 23

tformat_configureCLBLen (*C function*), 37

tformat_configureSPILen (*C function*), 37

tformat_exCommands (*C function*), 22

tformat_getCRC (*C function*), 40

tformat_getCRCID0_1_7_8_C (*C function*), 38

tformat_getCRCID2 (*C function*), 38

tformat_getCRCID3 (*C function*), 38

tformat_getCRCID6 (*C function*), 39

tformat_getCRCIDD (*C function*), 39

tformat_getRxCRCbyCLB (*C function*), 39

tformat_getTxCRCbyVCRC (*C function*), 39

tformat_init (*C function*), 22

tformat_initCLBXBAR (*C function*), 37

tformat_initSPIFIFO (*C function*), 22

tformat_resetCLB (*C function*), 37

tformat_spiRxISR (*C function*), 23

tformat_testEEPROMCommands (*C function*), 23

tformatData (*C var*), 19, 28

tformatRxData (*C var*), 20, 29

tformatSpiRxIsrTicker (*C var*), 21